Weather Data Ingestion Pipeline — Summary Report

What this project does

We built a Python script that automatically: 1. Picks 15 Toronto postal codes from the course database (`mmai_db`). 2. Calls a weather API (OpenWeather) to get the current weather for each postal code. 3. Cleans up that data and adds one extra useful field. 4. Saves everything into a table (`uploads.current_weather`) in our own database.

This is called an **ETL pipeline** — **E**xtract data, **T**ransform it, **L**oad it into a database.

Steps we followed

1. Extract (get the postal codes) We wrote a SQL query (`sql/extract_postal_codes.sql`) that selects 15 Toronto postal codes, along with their province, region, latitude, and longitude, from `mmai_db`.

2. Extract (get the weather) For each postal code, the script sends the latitude/longitude to the OpenWeather API and gets back the current temperature, "feels like" temperature, humidity, pressure, and a short weather description.

3. Transform (clean the data) Raw API data isn't ready to store as-is, so the script: - Converts the weather API's timestamp (a number like `1721923200`) into a normal, readable date/time. - Rounds temperature and other numbers to 2 decimal places. - Skips any postal code where the API didn't return usable data (instead of crashing). - Cleans up the weather description text (e.g. "clear sky" → "Clear Sky").

4. Transform (feature engineering) We also created one new column that isn't in the raw API data: `temp_category`. It buckets the temperature into a simple label — `freezing`, `cold`, `mild`, `warm`, or `hot` — which is easier to use in reports than a raw number.

5. Load (save to our database) The cleaned data is saved into `uploads.current_weather` in our own personal database (a *different* database from `mmai_db` — more on this below).

Key design decisions

- **Two databases, not one.** `mmai_db` (where the postal codes live) and our personal database (where we save weather data) are two separate databases on the same server. The script connects to both separately — one connection to *read* postal codes, one connection to *write* weather data.
- **No duplicate data.** Each row is uniquely identified by postal code + the exact time the weather was measured. If we run the pipeline twice, it won't create duplicate rows — it just skips anything already saved. This is what makes it safe to run every day.
- **Built to scale.** The pipeline works the same way whether it processes 15 postal codes or all postal codes in Canada — nothing in the code depends on the number 15. To scale up, we would only need to change the SQL query, not the Python script.

How we handle errors

- **Temporary network hiccups → retry automatically.** A timeout, "server busy" (5xx), or "too many requests" (429) triggers an automatic retry with increasing waits (2s, 4s, 8s... up to 5 tries) before giving up.

- **Invalid request → don't retry.** A bad API key (401) or similar client error won't fix itself by retrying, so the script logs it and moves on immediately.
- **One postal code fails → the rest keep going.** A single postal code's data being missing or unfetchable only skips that one row (logged as a warning); the other 14 still get saved. Critical once this scales to thousands of postal codes.
- **Database unreachable → the whole run stops.** Unlike a single postal code failing, a broken database connection can't be safely "skipped," so the pipeline stops and reports it.

Regular runs vs. backfill runs (and what a production version would add)

- **Regular run:** the normal daily scheduled run — captures whatever the weather is *right now* for all 15 postal codes, on schedule.
- **Catch-up run (what we support today):** if the daily run fails, we can simply run the exact same script by hand. Because saved data is never duplicated or overwritten, this is always safe. But right now nothing *tells* us a run failed — we'd only notice by checking the table ourselves.
- **True backfill (a real production concept, not built here):** in a production system, "backfill" usually means re-fetching the *correct* data for a specific past date after the fact. We can't do that with this pipeline, because OpenWeather's free endpoint only returns live conditions, not historical weather for a past date — a paid historical-data API would be needed to support real backfills.
- **How this would be monitored in production:** a real deployment would pair the daily job with a monitoring tool (e.g. Datadog, or even a simple Slack/email alert) that checks "did today's run insert any rows?" and pages someone if not — instead of relying on a person noticing a gap in the table days later. If that alert fires, an engineer investigates (expired API key, database down, etc.), fixes the root cause, then re-runs the script by hand — which is safe thanks to the idempotent load described above.

This project implements the pipeline itself and the safety net that makes retries/catch-up runs non-destructive; production-grade monitoring and true historical backfill are natural next steps beyond this assignment's scope.

Challenges faced

The main challenge was realizing `mmai_db` and our personal database are two *separate* databases, not one — we had to set up two database connections instead of one. The rest of the tricky parts (messy API data, avoiding duplicate rows) are covered above under "How we handle errors" and "Key design decisions."

How to schedule this to run daily

The script is a normal `.py` file, so it can be scheduled with standard tools — no code changes needed: - **Mac/Linux:** a `cron` job, e.g. `0 7 * * * python current_weather_pipeline.py` runs it every day at 7 AM. - **Windows:** Windows Task Scheduler, pointed at the same command. - **Cloud option:** a scheduled cloud job (e.g. Apache Airflow, GitHub Actions on a schedule, or a cloud provider's cron service) if we later want it to run without our laptop being on.

Because the pipeline never overwrites or duplicates existing data, it's safe to schedule it to run automatically every day without any manual checking.